Programming and databases

Joern Ploennigs

Operators

MIDJOURNEY: CALIGRAPHIC EQUATIONS

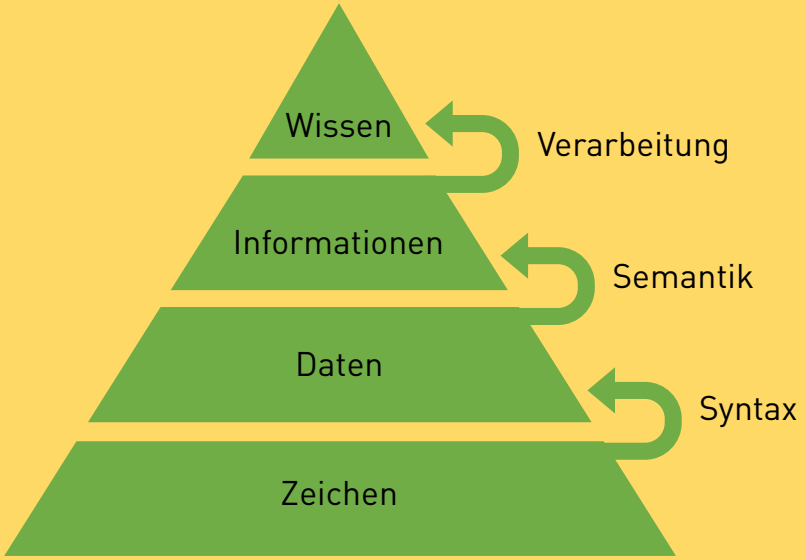
Recap: In-Class Question

What levels does the knowledge pyramid have?

Midjourney: Tower of Babel

Review: Knowledge Pyramid

The knowledge pyramid is a model for representing the emergence of knowledge.
The four element types: *Signs, Data, Information,* and *Knowledge* are represented as four levels of a pyramid.
Signs form the base and knowledge the apex of the pyramid.



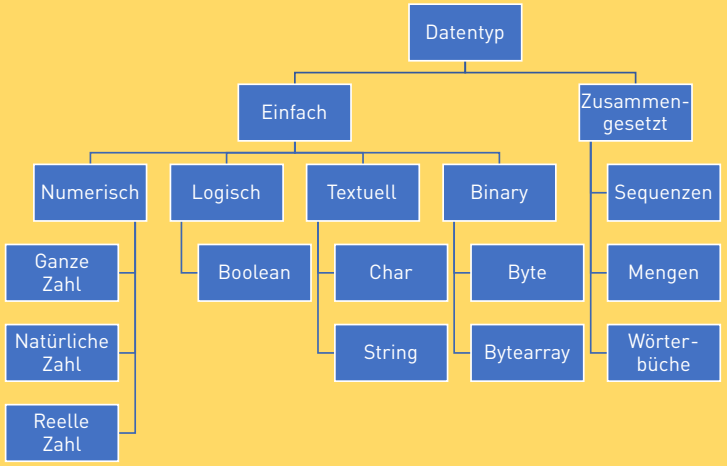
Review: Lecture Hall Question

What data types are there in programming languages?

DALL·E 2: A chair in the shape of a avocado

Wiederholung: Datentyp

The data type specifies what kind of data it describes (data contract) and which operations can be performed on it.



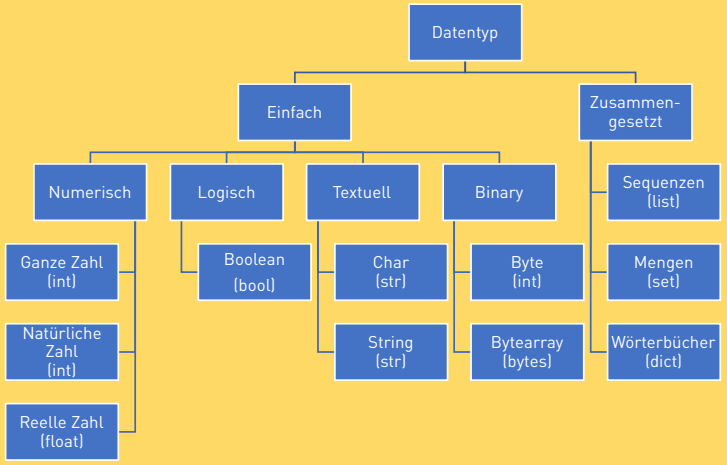
Review: In-class Question

What data types are there in Python?

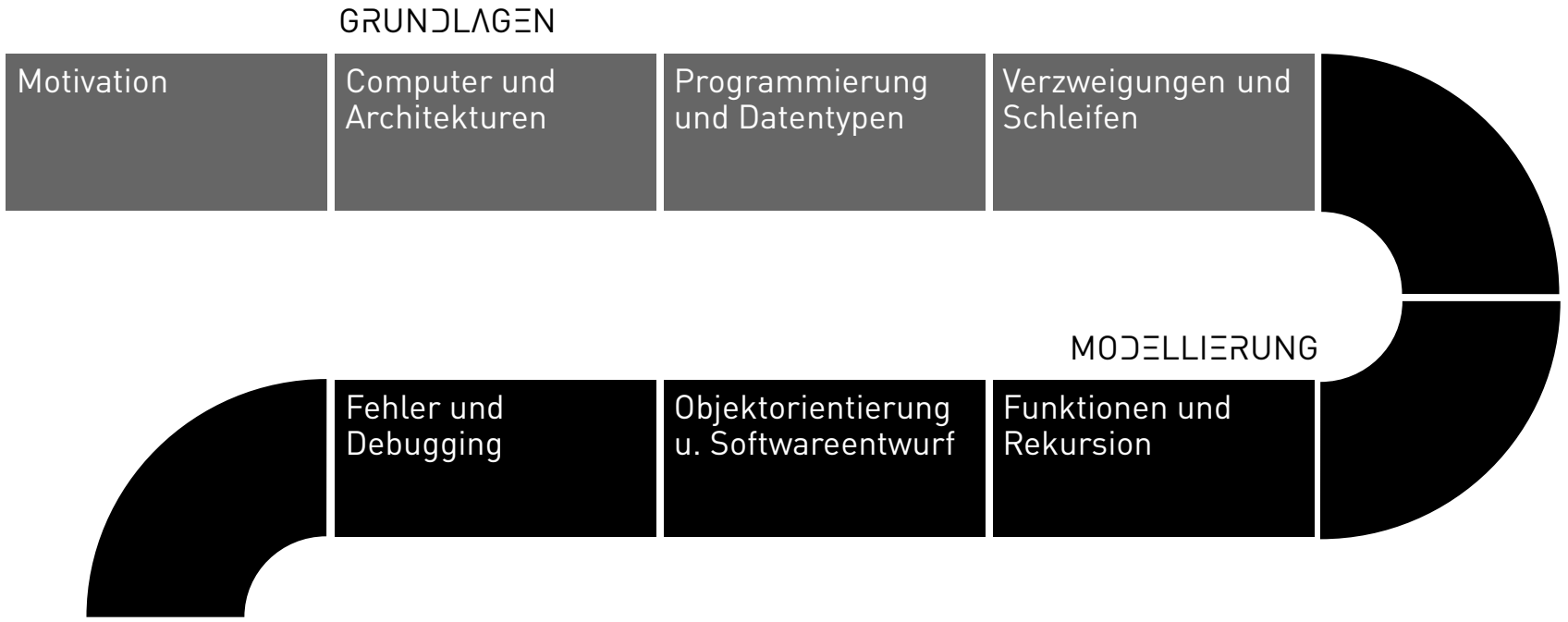
Midjourney: A python programming a robot

Recap: Data Types - Python

Python simplifies some data types.



Process


GRUNDLAGEN
Motivation
Computer und Architekturen
Programmierung und Datentypen
Verzweigungen und Schleifen
MODELLIERUNG
Funktionen und Rekursion
Objektorientierung u. Softwareentwurf
Fehler und Debugging

Instructions

- 📘 **Definition: Statement**
  A statement is a single instruction formulated in the syntax of a programming language that is executed sequentially in the program.

- 📘 **Definition: Expressions**
  A special form of statements are expressions, which always return a value and thus have a data type.

- 📘 **Definition: Operator**
  An operator is a part of expressions and is a symbol or keyword that tells the compiler or interpreter to perform a specific operation on values.

- Example: In a simple variable assignment, the expression appears to the right of the assignment operator `=`.

```
VariableName = Expression
```

Arithmetic Operators

Arithmetic operators perform the standard mathematical operations. They are used with numeric values to perform general mathematical operations.

Operator	Example	Effect
Addition	`x + y`	`10 + 3` (Result: 13)
Subtraction	`x - y`	`10 - 3` (Result: 7)
Multiplication	`x * y`	`10 * 3` (Result: 30)
Division	`x / y`	`10 / 3` (Result: 3.33)
Integer Division	`x // y`	`10 // 3` (Result: 3)
Modulo Operator	`x % y`	`10 % 3` (Result: 1)
Exponentiation	`x ** y`	`10 ** 3` (Result: 1000)

Assignment Operators

Assignment operators assign values to variables. Python uses augmented assignment for all arithmetic operations; however, there is no increment or decrement like in other languages.

Operator	Example	Effect
Assignment	`x = y`	Variable `x` receives the value `y`
Augmented addition	`x += 4`	`x = x + 4` # Variable `x` is increased by 4
Augmented multiplication	`x *= 4`	`x = x * 4` # Variable `x` is multiplied by 4
Augmented division	`x /= 4`	`x = x / 4` # Variable `x` is divided by 4
Augmented floor division	`x //= 4`	`x = x // 4` # Variable `x` is floor-divided by 4
Augmented modulo	`x %= 4`	`x = x % 4` # The remainder of dividing variable `x` by 4

Comparison Operators

Comparison operators compare two values and return true or false as a result. For example, `a == b` tests whether `a` is equal to `b`, and returns true or false accordingly, while `a != b` tests whether `a` is not equal to `b`.

Operator	Example	Effect
Equals	`x == y`	true if `x` is equal to `y`
Not equal	`x != y`	true if `x` is not equal to `y`
Greater than	`x > y`	true if `x` is greater than `y`
Less than	`x < y`	true if `x` is less than `y`
Greater than or equal to	`x >= y`	true if `x` is greater than or equal to `y`
Less than or equal to	`x <= y`	true if `x` is less than or equal to `y`

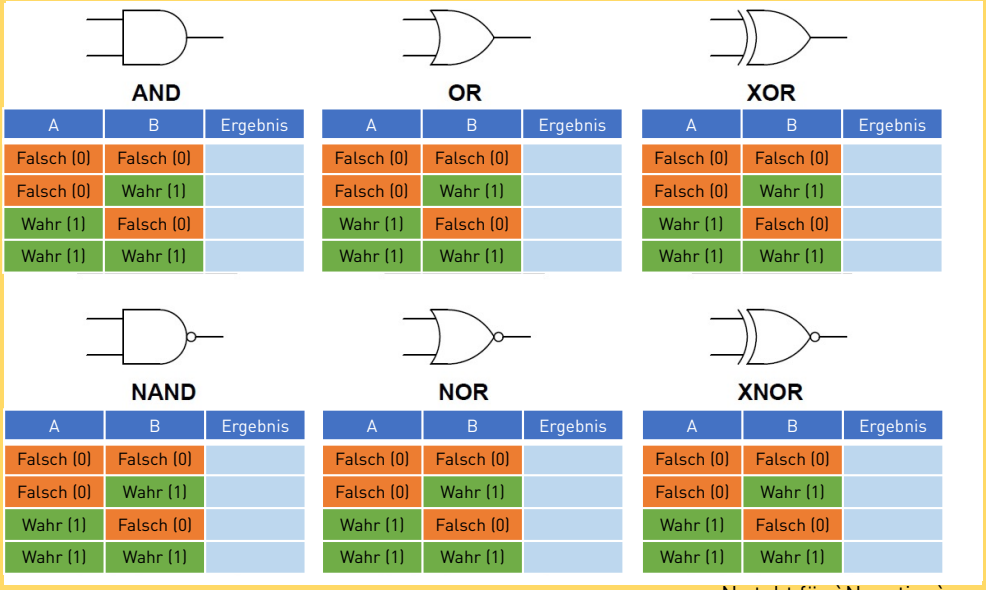
Identity Operators

Identity operators compare objects. For example, if `x = [1, 2, 3]` holds, i.e., `x` is a list, then `type(x) is list` evaluates to true, and `type(x) is not list` is false. They check whether two objects refer to the same memory location. `x is y` returns true when both variables refer to the same object, i.e., point to the same memory location.

Operator	Example	Effect
Is	`x is y`	true if both variables refer to the same object.
Is not	`x is not y`	true if the variables do not refer to the same object.

Lecture hall question

What outcomes do you get for each logical operation?
(Corresponds to propositional logic)



AND

A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	

OR

A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	

XOR

A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	

NAND

A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	

NOR

A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	

XNOR

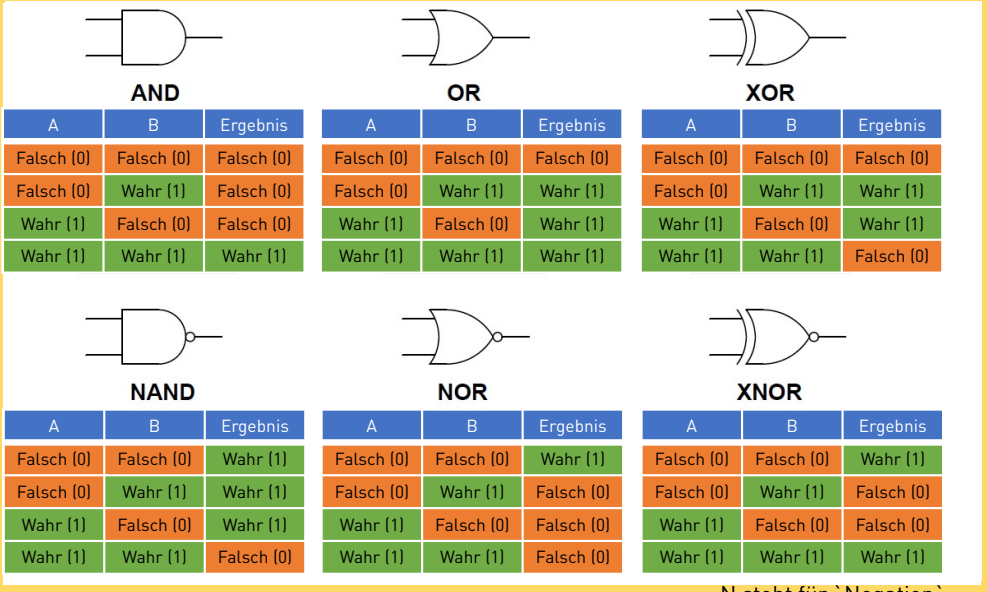
A	B	Ergebnis
Falsch (0)	Falsch (0)	
Falsch (0)	Wahr (1)	
Wahr (1)	Falsch (0)	
Wahr (1)	Wahr (1)	

N steht für `Negation`

Logical operations

Lecture Hall Question

What results are there for each logical operation?
(Corresponds to propositional logic)



AND

A	B	Ergebnis
Falsch (0)	Falsch (0)	Falsch (0)
Falsch (0)	Wahr (1)	Falsch (0)
Wahr (1)	Falsch (0)	Falsch (0)
Wahr (1)	Wahr (1)	Wahr (1)

OR

A	B	Ergebnis
Falsch (0)	Falsch (0)	Falsch (0)
Falsch (0)	Wahr (1)	Wahr (1)
Wahr (1)	Falsch (0)	Wahr (1)
Wahr (1)	Wahr (1)	Wahr (1)

XOR

A	B	Ergebnis
Falsch (0)	Falsch (0)	Falsch (0)
Falsch (0)	Wahr (1)	Wahr (1)
Wahr (1)	Falsch (0)	Wahr (1)
Wahr (1)	Wahr (1)	Falsch (0)

NAND

A	B	Ergebnis
Falsch (0)	Falsch (0)	Wahr (1)
Falsch (0)	Wahr (1)	Wahr (1)
Wahr (1)	Falsch (0)	Wahr (1)
Wahr (1)	Wahr (1)	Falsch (0)

NOR

A	B	Ergebnis
Falsch (0)	Falsch (0)	Wahr (1)
Falsch (0)	Wahr (1)	Falsch (0)
Wahr (1)	Falsch (0)	Falsch (0)
Wahr (1)	Wahr (1)	Falsch (0)

XNOR

A	B	Ergebnis
Falsch (0)	Falsch (0)	Wahr (1)
Falsch (0)	Wahr (1)	Falsch (0)
Wahr (1)	Falsch (0)	Falsch (0)
Wahr (1)	Wahr (1)	Wahr (1)

N steht für `Negation`

Logical Operations

Logical Operators

Logical operators combine boolean expressions, which are then evaluated as true or false. For example, here the variable `is_teenager = (alter > 13) and (alter < 19)` is evaluated as true when the value of the variable alter is equal to 14.
`P` and `Q` stand for logical expressions, e.g. `P = (x > 10)` and `Q = (x < 20)`.

Operator	Example	Effect
Logical AND	`P and Q`	Returns true (1) if both expressions `P` and `Q` are true.
Logical OR	`P or Q`	Returns true (1) if either expression `P` or `Q` is true.
Logical NOT	`not P`	Negates the expression `P`: true becomes false, false becomes true.

Bitwise Operators

Bitwise operators perform bitwise operations on integer operands. In other words, the operands are converted to their binary representations, e.g., `x = 5 = 0101`, `y = 3 = 0011`, and then each bit in operand `x` is combined with the bit at the corresponding position in operand `y`.
| Operator | Example | Effect | | --- | --- | --- | | Bitwise AND | `x & y` | The resulting bit is 1 if both bits are 1. | | Bitwise OR | `x | y` | The resulting bit is 1 if at least one of the bits is 1. | | Bitwise XOR (Exclusive OR) | `x ^ y` | The resulting bit is 1 if exactly one of the bits is 1. | | Bitwise NOT | `~x` | Returns the one's complement of `x`. | | Right shift | `x >> n` | The bits in `x` are shifted to the right by `n` positions. | | Left shift | `x << n` | The bits in `x` are shifted to the left by `n` positions. |

Membership operators

Membership operators check whether an object is a member of a list, e.g., `4 in [1,2,3]` checks whether 4 is contained in the list.
`x` here stands for an arbitrary object, and `list` for a collection.

Operator	Example	Effect
in	`x in list`	True if x is in the list `list`.
not in	`x not in list`	True if x is not in the list `list`.

Questions?

PROGRAMMIERUNG UND DATENBANKEN